

PROYECTO INTEGRADOR

Sistema de Administración de un Club Deportivo

Análisis, diseño, desarrollo, prueba y puesta a punto

Tecnología: C# · Windows Forms · MySQL

Arquitectura en capas: Presentación · Negocio · Datos · Entidades

Índice

Índice	2
1. Enunciado del requerimiento del cliente	4
1.1 Reglas de negocio relevadas	4
1.2 El sistema debe permitir controlar	4
1.3 Entregables solicitados	4
2. Estrategia de solución	5
3. Lista de variables	6
Persona (clase base abstracta)	6
Socio	6
Cuota	6
Profesor / Actividad / Sueldo / AsistenciaProfesor	6
ConsultaNutricion / Usuario	6
Fase 1 — Análisis preliminar	7
Salidas del sistema	7
Entradas de datos	7
Límite y objetivo del sistema	7
Módulos del sistema (desagregación del objetivo)	7
Fase 2 — Análisis previo y funcional	8
Procesos del sistema	8
Clases con atributos que intervienen	8
Redacción lógica del proceso central: Cobrar Cuota	8
Boceto de pantallas de la interfaz	8
Fase 3 — Análisis orgánico	10
Diálogo del sistema con el usuario	10
Cómo se informan errores de datos	10
Fase 4 — Conectar datos para programar	11
Cómo se enlazan los datos con el código	11
Control de acceso de usuarios	11
Fase 5 — Programar	12
Codificación de pantallas	12
Consultas a la base de datos	12
Fase 6 — Prueba	13
Quién prueba el sistema	13
Cómo reiniciar el sistema si hay errores	13
Fase 7 — Puesta a punto	14
Cómo crear el ejecutable (Release)	14
Cómo se implementa el sistema	14
4. Diagrama de Clases	15
5. Diagramas de Casos de Uso	16
5.1 Diagrama de Secuencia — Cobrar Cuota (opcional)	17
5.2 Diagrama de Actividades — Alta de Socio (opcional)	18

6. Juego de prueba	19
7. Conclusiones del desarrollo	20
8. Instrucciones paso a paso	21
8.1 Crear el proyecto en Visual Studio	21
8.2 Crear la carpeta "Datos" dentro del proyecto	21
8.3 Instalar el conector MySQL (NuGet MySql.Data)	21
8.4 Configurar la base de datos	21
8.5 Ejecutar y compilar	21
8.6 Generar el ejecutable (Release) y desplegarlo	21

1. Enunciado del requerimiento del cliente

La empresa administra un **club deportivo** que ofrece actividades tanto para **socios** como para **no socios**, y solicita un sistema de escritorio (Windows Forms) con base de datos MySQL que informatice la gestión integral del club. A continuación, se transcribe, organizado, el enunciado relevado del cliente.

1.1 Reglas de negocio relevadas

- Los socios abonan una **cuota mensual**; los no socios abonan únicamente el costo de la actividad que realizan en el momento, según cartilla de precios.
- La cuota se abona **por adelantado**, en efectivo o con tarjeta de crédito. Con tarjeta existen promociones en 3 y 6 cuotas.
- La inscripción por primera vez al gimnasio implica completar una ficha y presentar el apto físico.
- El club posee un plantel propio de **profesores** y un listado de **suplentes**.
- Vencido el período de pago, el socio automáticamente **no puede realizar actividades**. Para retomar debe abonar la cuota; el nuevo plazo comienza a correr desde el día siguiente al vencimiento.
- Cada profesor, además de dictar su clase, controla el salón general (musculación y aparatos) y confecciona rutinas para sus alumnos.
- Los profesores firman planilla de asistencia al llegar al club.
- El sueldo de los profesores se liquida mensualmente, el último día hábil del mes.
- El servicio de **nutrición** funciona una vez por semana, con turno asignado con anticipación; se lleva ficha médica de cada socio y se establece la carga de actividad física permitida.
- Se entrega un **carnet** al socio.

1.2 El sistema debe permitir controlar

1. Registro de socios y no socios.
2. Entrega de carnet y cobro de cuota mensual o diaria.
3. Listado diario de los socios a quienes, en la fecha, les vence la cuota.

1.3 Entregables solicitados

- Enunciado del requerimiento y estrategia de solución.
- Lista de variables con su significado.
- Diagrama de Clases y Diagramas de Casos de Uso
- Juego de prueba con resultados esperados.
- Código fuente en C# (Windows Forms + MySQL), organizado en capas, listo para ejecutar.
- Conclusiones del desarrollo y documento final con numeración de hojas.

2. Estrategia de solución

Se optó por una **arquitectura en capas** en C# / Windows Forms, que separa responsabilidades y facilita el mantenimiento y las pruebas:

Capa	Carpeta	Responsabilidad
Presentación	Formularios/	14 formularios Windows Forms (ABM, cobros, listados). Validan datos de entrada y delegan toda regla de negocio en la capa de Negocio.
Negocio	Negocio/	Reglas del club: cuota al día, promociones con tarjeta, turno de nutrición semanal, último día hábil, etc. No conoce SQL ni controles de UI.
Datos	Datos/	Clase Conexion (abre/cierra la conexión a MySQL) y una clase DAO por entidad, con consultas parametrizadas y, en operaciones críticas, stored procedures.
Entidades	Entidades/	Clases POCO (Persona abstracta, Socio, Profesor, Cuota, etc.) que representan el modelo de datos en memoria.

La carpeta **Datos** concentra todo lo relacionado con el entorno de datos, tal como pide la consigna: la clase Conexion (con métodos Abrir() / Cerrar(), bloques try/catch/finally y cadena de conexión configurable desde App.config) y las 13 clases DAO. Todas las consultas están parametrizadas (SqlParameter / AddWithValue) para evitar inyección SQL, y las operaciones críticas — alta de socio, cobro de cuota y listado de vencimientos — se resuelven con **stored procedures** (CommandType.StoredProcedure), mientras que las demás usan CommandType.Text.

Decisiones de diseño relevantes:

- **Persona** es una clase abstracta de la que heredan Socio, NoSocio y Profesor (evita duplicar nombre, DNI, contacto, etc.). En la base de datos se refleja como una tabla persona con las tablas socio, profesor y no_socio relacionadas por id_persona (claves foránea).
- El estado de la cuota **no se guarda como bandera manual**: se calcula comparando la fecha de hoy contra el último vencimiento pagado (SocioNegocio.PuedeRealizarActividad), así nunca queda desincronizado.
- Las contraseñas de usuario se guardan como **hash SHA-256**, nunca en texto plano.
- El acceso está controlado por **rol** (Administrador, Recepcionista, Profesor, Nutricionista): frmMenuPrincipal arma el menú según el rol del usuario autenticado.

3. Lista de variables

Se listan las variables más relevantes del modelo, agrupadas por entidad. Los nombres coinciden con las propiedades de las clases en Entidades/ y con las columnas de la base Proyecto.

Persona (clase base abstracta)

Variable	Tipo	Significado
id	int	Identificador único de la persona (PK de persona).
nombre / apellido	string	Nombre y apellido de la persona.
dni	string	Documento de identidad, único en el sistema.
fechaNacimiento	date	Fecha de nacimiento; usada para calcular la edad.
telefono / direccion / email	string	Datos de contacto.

Socio

Variable	Tipo	Significado
nroSocio	string	Número de socio, único, impreso en el carnet.
fechaAlta	date	Fecha de ingreso al club.
aptoFisico	bool	Indica si presentó el apto físico (requisito para carnet y 1ª vez en el gimnasio).
estado	string	'Activo' / 'Inactivo' (baja lógica).
carnetEntregado	bool	Indica si ya se le emitió el carnet.
ultimoVencimiento	date?	(Calculada) fecha de vencimiento de la última cuota pagada.

Cuota

Variable	Tipo	Significado
mes / anio	int	Período que cubre la cuota.
monto	decimal	Importe abonado.
fechaVencimiento	date	Fecha hasta la cual el socio puede realizar actividades.
fechaPago	date	Fecha real en que se realizó el pago.
formaPago	string	'Efectivo' o 'Tarjeta'.
cuotasTarjeta	int?	3 o 6 si la forma de pago es Tarjeta; null en caso contrario.

Profesor / Actividad / Sueldo / AsistenciaProfesor

Variable	Tipo	Significado
legajo	string	Identificador interno del profesor, único.
tipo (Profesor)	string	'Titular' o 'Suplente'.
horariosAsignados	string	Franjas horarias en las que dicta clase / controla el salón.
costoNoSocio	decimal	Precio de la actividad para quien no es socio (cartilla).
cupoMaximo	int	Cantidad máxima de inscriptos a la actividad.
horaEntrada	time	Hora en que el profesor firmó la asistencia del día.
monto (Sueldo)	decimal	Importe liquidado en el período mes/año.

ConsultaNutricion / Usuario

Variable	Tipo	Significado
turno	int	Número de orden del turno del día (el servicio atiende una vez por semana).
cargaActividadPermitida	string	Actividad física habilitada, definida junto al nutricionista.

Variable	Tipo	Significado
nombreUsuario / contraseñaHash	string	Credenciales de acceso; la contraseña se guarda con hash SHA-256.
rol	string	'Administrador' 'Recepcionista' 'Profesor' 'Nutricionista'.

Fase 1 — Análisis preliminar

Salidas del sistema

- Carnet de socio (pantalla + registro en BD).
- Listado diario de vencimientos de cuota.
- Comprobante / confirmación de cobro de cuota.
- Listados: socios activos/inactivos, recaudación del mes, carnets emitidos, ocupación de actividades.
- Planilla de asistencia de profesores.
- Liquidación de sueldos del período.

Entradas de datos

- Datos personales de socios, no socios y profesores.
- Datos de la actividad (nombre, costo, horario, cupo, profesor a cargo).
- Datos del cobro de cuota (monto, forma de pago, cuotas de tarjeta).
- Ficha médica / turno de nutrición.
- Firma de asistencia diaria del profesor.
- Credenciales de acceso (usuario / contraseña).

Límite y objetivo del sistema

El sistema **abarca** la gestión administrativa del club: altas de personas, cobro de cuotas y de actividades, control de acceso por cuota al día, planteles de profesores, rutinas, asistencia, sueldos y turnos de nutrición. **No abarca** facturación fiscal (AFIP), pasarela de pago real con tarjeta (se registra la forma de pago, no se procesa la transacción financiera) ni control biométrico de acceso físico.

Módulos del sistema (desagregación del objetivo)

1. Login y control de acceso por rol
2. Gestión de Socios
3. Gestión de No Socios
4. Gestión de Profesores (titulares y suplentes)
5. Gestión de Actividades
6. Gestión de Cuotas (cobro, formas de pago, promociones)
7. Listado diario de vencimientos
8. Entrega de Carnet
9. Control de acceso a actividades (cuota al día)
10. Gestión de Rutinas
11. Asistencia de Profesores
12. Liquidación de Sueldos
13. Consultas de Nutrición (turnos, ficha médica)
14. Reportes y listados

Fase 2 — Análisis previo y funcional

Procesos del sistema

- Alta de socio (con validación de DNI y N° de socio únicos).
- Cobro de cuota mensual (efectivo / tarjeta, con recálculo del vencimiento).
- Registro de no socio y cobro de la actividad puntual.
- Emisión de carnet (sujeta a apto físico y cuota al día).
- Listado diario de vencimientos.
- Alta de profesor, firma de asistencia y liquidación de sueldo.
- Alta y consulta de rutinas.
- Asignación de turnos de nutrición (solo miércoles).

Clases con atributos que intervienen

Ver el **Diagrama de Clases** (sección 5) y la **Lista de variables** (sección 3): Persona, Socio, NoSocio, Profesor, Actividad, Cuota, Inscripcion, Carnet, Rutina, AsistenciaProfesor, Sueldo, ConsultaNutricion, Usuario.

Redacción lógica del proceso central: Cobrar Cuota

1. La recepcionista busca al socio por N° de socio o DNI.
2. El sistema muestra nombre, N° de socio y el último vencimiento pagado.
3. La recepcionista ingresa monto, forma de pago (Efectivo/Tarjeta) y, si es Tarjeta, la promoción (3 o 6 cuotas).
4. CuotaNegocio valida: monto > 0; forma de pago válida; promoción solo 3 o 6 con Tarjeta.
5. CuotaDAO ejecuta sp_CobrarCuota:
 - SI el socio nunca pagó -> nuevo vencimiento = hoy + 1 mes
 - SI ya tiene un vencimiento -> nuevo vencimiento = (último vencimiento + 1 día) + 1 mes
6. Se inserta la cuota y se marca el socio como 'Activo'.
7. El sistema informa la nueva fecha de vencimiento. Si, aun así, quedó en el pasado (mucho atraso), se avisa que el socio sigue vencido y debe abonar los períodos adeudados.

Boceto de pantallas de la interfaz

A continuación, se muestra el boceto de cada formulario (los 14 detallados en la Fase 5).

Fig. 1 — Boceto de frmLogin

Gestión de Socios

Nombre

Apellido

DNI

Fecha de nacimiento

Teléfono

Dirección

Email

N° de Socio

☐ Apto físico presentado

☐ Carnet entregado

Estado

Nuevo

Guardar

Modificar

Eliminar

Buscar

Salir

N° Socio	Apellido	Nombre	DNI	Estado	Cuota al día

Archivo: Formularios/frmSocios.cs

Fig. 2 — Boceto de frmSocios (ABM)

Cobro de Cuotas

N° de socio o DNI

Monto a cobrar (\$)

Forma de pago

Promoción con tarjeta (3/6 cuotas)

Buscar

Cobrar cuota

Salir

Mes	Año	Monto	Vence	Fecha de pago	Forma de pago	Cuotas

Archivo: Formularios/frmCuotas.cs

Muestra datos del socio, estado de la cuota e historial.

Fig. 3 — Boceto de frmCuotas

Fase 3 — Análisis orgánico

Diálogo del sistema con el usuario

- Cada ABM usa siempre los mismos seis botones (**Nuevo, Guardar, Modificar, Eliminar, Buscar, Salir**) para que el usuario reconozca el patrón en cualquier pantalla.
- Las acciones irreversibles (baja de socio, eliminación de actividad/rutina, cancelación de turno) piden **confirmación** (`UIHelper.Confirmar`) antes de ejecutarse.
- Toda operación exitosa muestra un mensaje de confirmación (`UIHelper.Info`), por ejemplo, la nueva fecha de vencimiento al cobrar una cuota.
- `frmMenuPrincipal` arma el menú según el **rol** del usuario autenticado, de modo que cada perfil solo ve las opciones que le corresponden.

Cómo se informan errores de datos

Toda validación pasa por `UIHelper` antes de tocar la base de datos: `Requerido`, `ComboRequerido`, `Decimal` y `Entero` muestran un `MessageBox` de error, ubican el foco en el campo inconsistente y **cancelan** la operación. Los errores de acceso a datos (conexión caída, violación de una clave única, motor no disponible) se capturan en cada DAO con `try/catch/finally`, se re-lanzan con un mensaje en español orientado al usuario y la capa de Presentación los muestra sin exponer detalles técnicos de SQL. La clase `Conexion` cierra siempre la conexión en el `finally`, incluso si la operación falló.

Fase 4 — Conectar datos para programar

Cómo se enlazan los datos con el código

Toda la conexión a MySQL vive en la carpeta **Datos/**, tal como pide la consigna:

1. En el Explorador de Soluciones de Visual Studio, clic derecho sobre el proyecto → **Agregar** → **Nueva carpeta** → nombrarla **Datos**.
2. Dentro de Datos/ se crea la clase **Conexion**, con `using MySql.Data.MySqlClient;` al comienzo del archivo.
3. Conexion expone `Abrir()` y `Cerrar()`, usa bloques `try/catch/finally` y arma la cadena de conexión leyendo `App.config` (clave `ClubDeportivoDB`), con un valor por defecto embebido si la clave no existiera.
4. Cada clase DAO (`SocioDAO`, `CuotaDAO`, etc.) instancia Conexion, abre, ejecuta su `MySqlCommand` parametrizado y cierra en el `finally`.

```
// Datos/Conexion.cs (fragmento)
using MySql.Data.MySqlClient;

public MySqlConnection Abrir()
{
    try
    {
        conexion = new MySqlConnection(ObtenerCadenaConexion());
        conexion.Open();
        return conexion;
    }
    catch (MySqlException ex)
    {
        throw new Exception("Error al conectar a la base de datos MySQL: " + ex.Message, ex);
    }
}
```

Cadena de conexión (entorno local, XAMPP / MySQL Workbench):

```
Server=localhost;Port=3306;Database=Proyecto;Uid=root;Pwd=;SslMode=none;
```

Control de acceso de usuarios

`frmLogin` solicita usuario y contraseña; `UsuarioNegocio.Login` calcula el hash SHA-256 de la contraseña ingresada y `UsuarioDAO.Authenticar` lo compara (parametrizado) contra `usuario.contrasena_hash`, exigiendo además `activo = 1`. Si coincide, se recupera el **rol** y `Program.cs` abre `frmMenuPrincipal`, que arma el menú disponible según ese rol (Administrador, Recepcionista, Profesor o Nutricionista).

Fase 5 — Programar

Codificación de pantallas

Se codificaron **14 formularios** en Formularios/, cada uno con el patrón estándar del Editor de Windows Forms de Visual Studio: una clase parcial dividida en ``FormName.cs`` (lógica y manejadores de evento) y ``FormName.Designer.cs`` (los controles, generados por `InitializeComponent()`), más su ``FormName.resx``. Esto permite abrir cualquier formulario en la **vista de Diseñador** y modificarlo arrastrando controles, igual que un formulario creado desde cero en Visual Studio. Las validaciones y mensajes comunes (campo obligatorio, número válido, confirmar baja) se centralizaron en `UIHelper`, usado desde el código de cada formulario:

Formulario	Función principal
frmLogin	Inicio de sesión (usuario / contraseña con hash SHA-256).
frmMenuPrincipal	Contenedor MDI; arma el menú según el rol del usuario.
frmSocios	ABM de socios: alta, baja lógica, modificación, búsqueda por DNI/N° socio.
frmNoSocios	Registro de la visita de un no socio y cobro de la actividad realizada.
frmProfesores	ABM de profesores, con filtro 'solo suplentes'.
frmActividades	ABM de actividades del club (costo, horario, cupo, profesor).
frmCuotas	Cobro de cuota mensual (Efectivo/Tarjeta, 3 o 6 cuotas) e historial del socio.
frmVencimientos	Requerimiento 3: listado diario de socios que vencen en una fecha.
frmCarnet	Emisión de carnet (valida apto físico y cuota al día).
frmRutinas	ABM de rutinas que cada profesor confecciona para sus alumnos.
frmAsistenciaProfesores	Firma de asistencia diaria y su historial.
frmSueldos	Liquidación mensual de sueldos, con aviso de último día hábil.
frmNutricion	Turnos de nutrición (solo miércoles) y ficha médica del socio.
frmReportes	Listados: socios, recaudación, carnets, ocupación de actividades.

Consultas a la base de datos

Todas las consultas se ejecutan mediante `MySqlCommand` con parámetros (`AddWithValue`), nunca concatenando texto del usuario dentro del SQL. Las operaciones críticas señaladas por la consigna — alta de socio, cobro de cuota y listado diario de vencimientos, además de la liquidación de sueldo — usan `CommandType.StoredProcedure`; el resto de los ABM usa `CommandType.Text`. Ejemplo:

```
var cmd = new MySqlCommand("sp_CobrarCuota", con) { CommandType = CommandType.StoredProcedure };
cmd.Parameters.AddWithValue("p_idsocio", idSocio);
cmd.Parameters.AddWithValue("p_monto", monto);
cmd.Parameters.AddWithValue("p_formapago", formaPago);
cmd.Parameters.AddWithValue("p_cuotastarjeta", (object)cuotasTarjeta ?? DBNull.Value);
DateTime nuevoVencimiento = Convert.ToDateTime(cmd.ExecuteScalar());
```

Fase 6 — Prueba

Quién prueba el sistema

En esta etapa académica, el propio desarrollador ejecuta el **juego de pruebas** de la sección 6 contra la base Proyecto recién creada (con los datos de ejemplo del script SQL), verificando cada caso contra el resultado esperado. En un contexto profesional, esta primera ronda la haría el equipo de desarrollo (pruebas unitarias e integración) y una segunda ronda la harían usuarios finales representativos de cada rol (recepción, profesores, nutrición) antes de la puesta en producción.

Cómo reiniciar el sistema si hay errores

- **Error de conexión a MySQL:** verificar que el servicio de MySQL/XAMPP esté iniciado y que los datos de App.config (servidor, puerto, usuario, clave) coincidan con la instalación local.
- **Error de datos (excepción de negocio):** la operación se cancela, se muestra el motivo y el formulario permanece con los datos cargados para corregir y reintentar; no se requiere reiniciar la aplicación.
- **Error inesperado no controlado:** cerrar y volver a abrir ClubDeportivo.exe; gracias al finally de Conexion.Cerrar(), no quedan conexiones abiertas colgadas en el motor.
- **Reinicio de datos de prueba:** volver a ejecutar Database/proyecto_club.sql (comienza con DROP DATABASE IF EXISTS Proyecto;), lo que recrea el esquema y los datos de ejemplo desde cero.

Fase 7 — Puesta a punto

Cómo crear el ejecutable (Release)

```
# Desde la carpeta que contiene ClubDeportivo.sln
dotnet publish ClubDeportivo\ClubDeportivo.csproj -c Release -r win-x64 --self-contained false

# o, desde Visual Studio:
# Compilar -> Configuration Manager -> Configuración: Release
# Compilar -> Publicar (o Compilar -> Compilar ClubDeportivo)
# El ejecutable queda en ClubDeportivo\bin\Release\net48\
```

Cómo se implementa el sistema

1. Instalar MySQL (XAMPP o MySQL Server + Workbench) en el equipo destino.
2. Ejecutar Database/proyecto_club.sql para crear la base Proyecto con sus tablas, procedimientos y datos de ejemplo.
3. Copiar la carpeta bin/Release/net48 (o el resultado de dotnet publish) al equipo destino.
4. Ajustar, si hace falta, la cadena de conexión del ClubDeportivo.exe.config (servidor, puerto, usuario, clave).
5. Ejecutar ClubDeportivo.exe e iniciar sesión con alguno de los usuarios de prueba (ver Fase 6 / Instrucciones).

4. Diagrama de Clases

Modelo de clases del dominio. Persona es abstracta; Socio, NoSocio y Profesor heredan de ella.

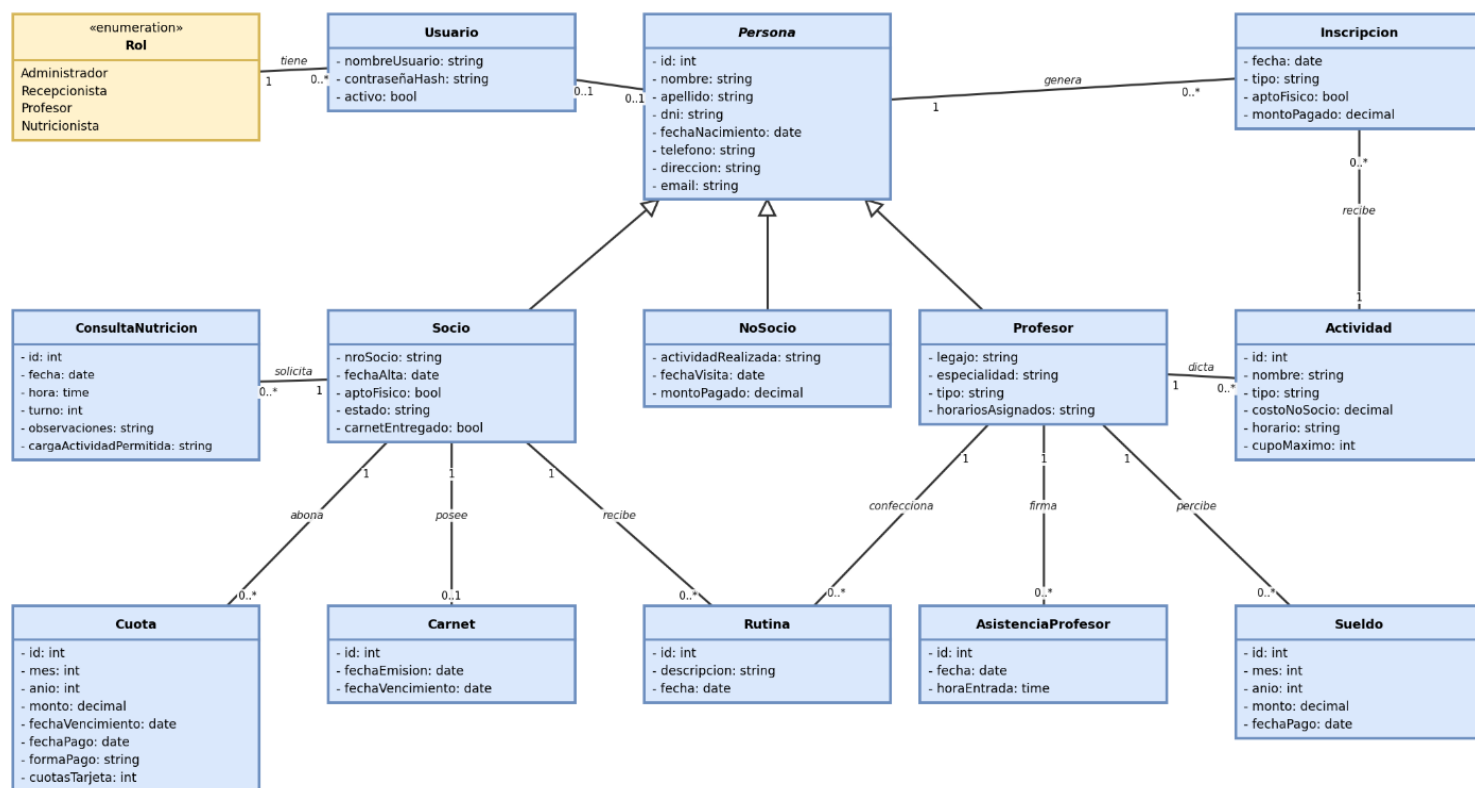


Fig. 4 — Diagrama de Clases (UML)

5. Diagramas de Casos de Uso

Actores: **Administrador, Recepcionista, Profesor, Nutricionista y Socio.**

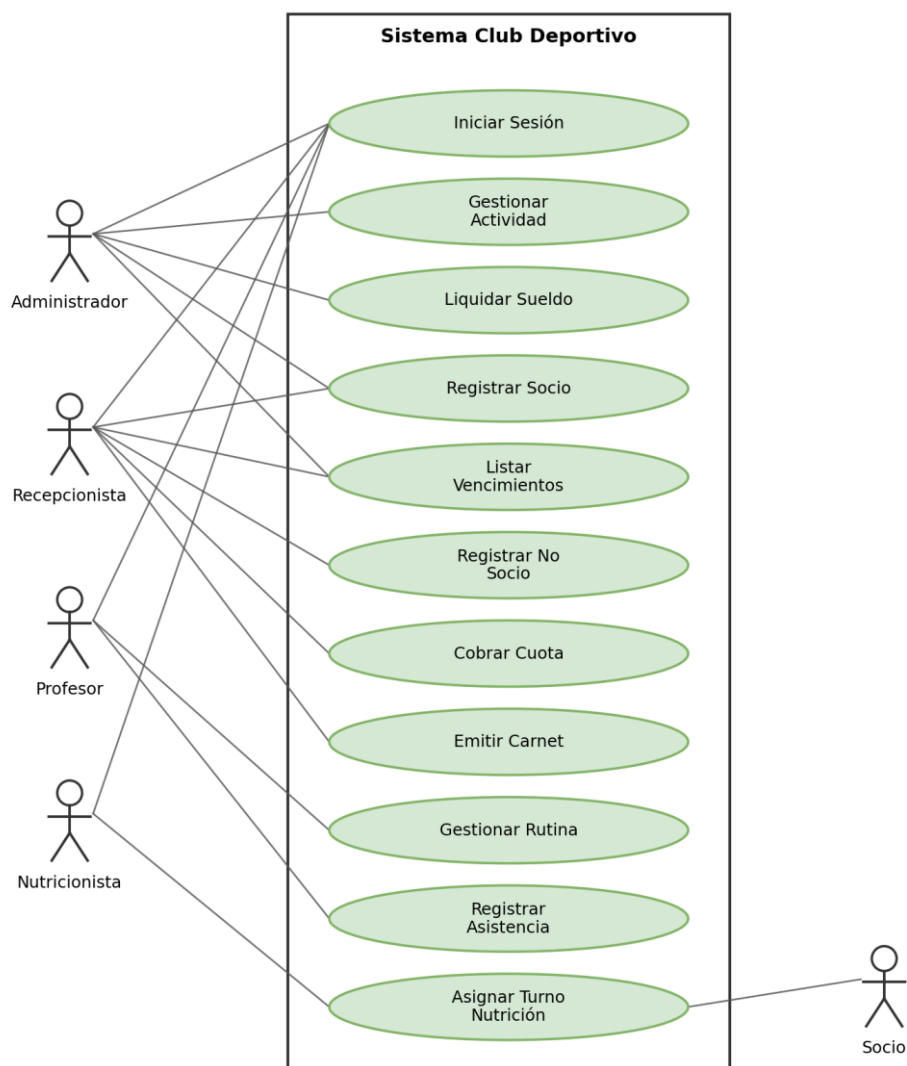


Fig. 5 — Diagrama de Casos de Uso

5.1 Diagrama de Secuencia — Cobrar Cuota (opcional)

Muestra el recorrido del caso de uso *Cobrar Cuota* entre la interfaz, la capa de negocio, el DAO y el stored procedure.

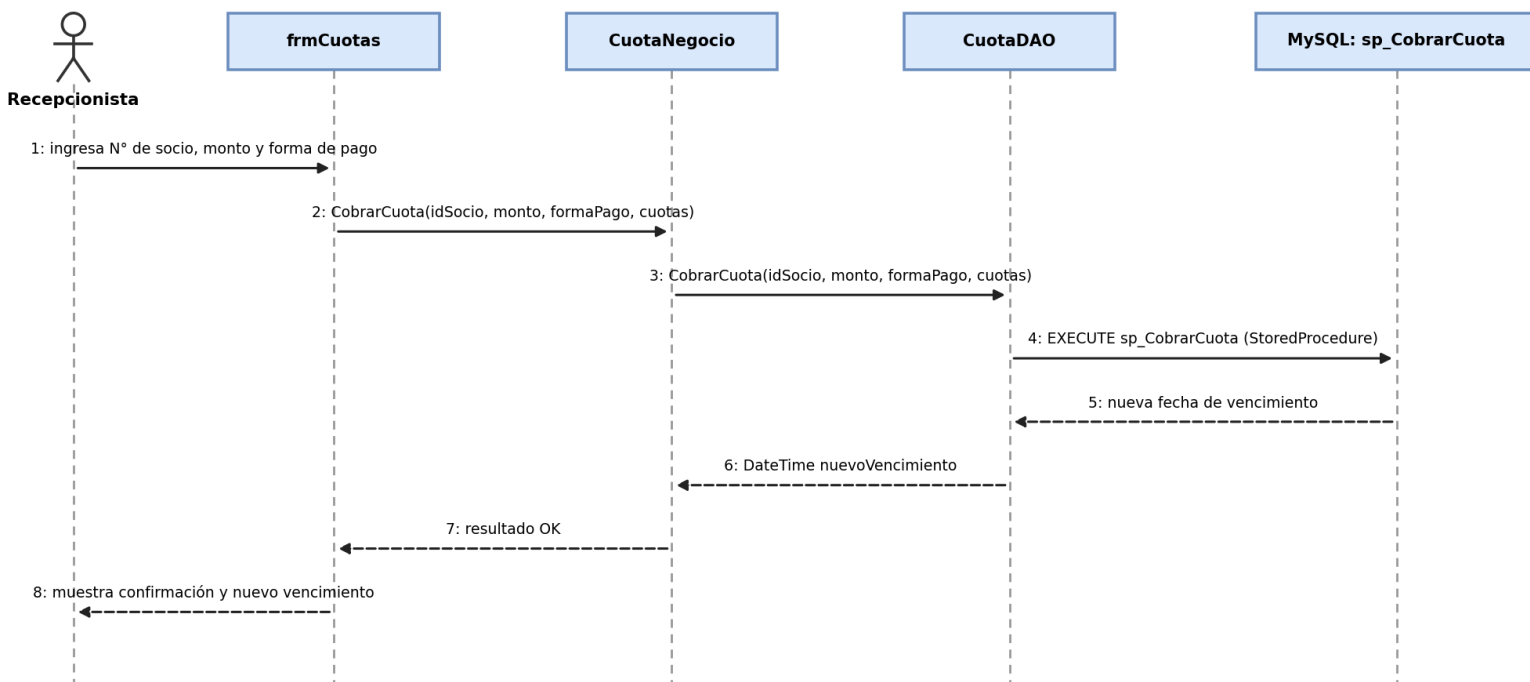


Fig. 6 — Diagrama de Secuencia: Cobrar Cuota

5.2 Diagrama de Actividades — Alta de Socio (opcional)

Modela el flujo completo del alta de un socio, incluyendo la ficha y el apto físico cuando es su primera vez en el gimnasio.

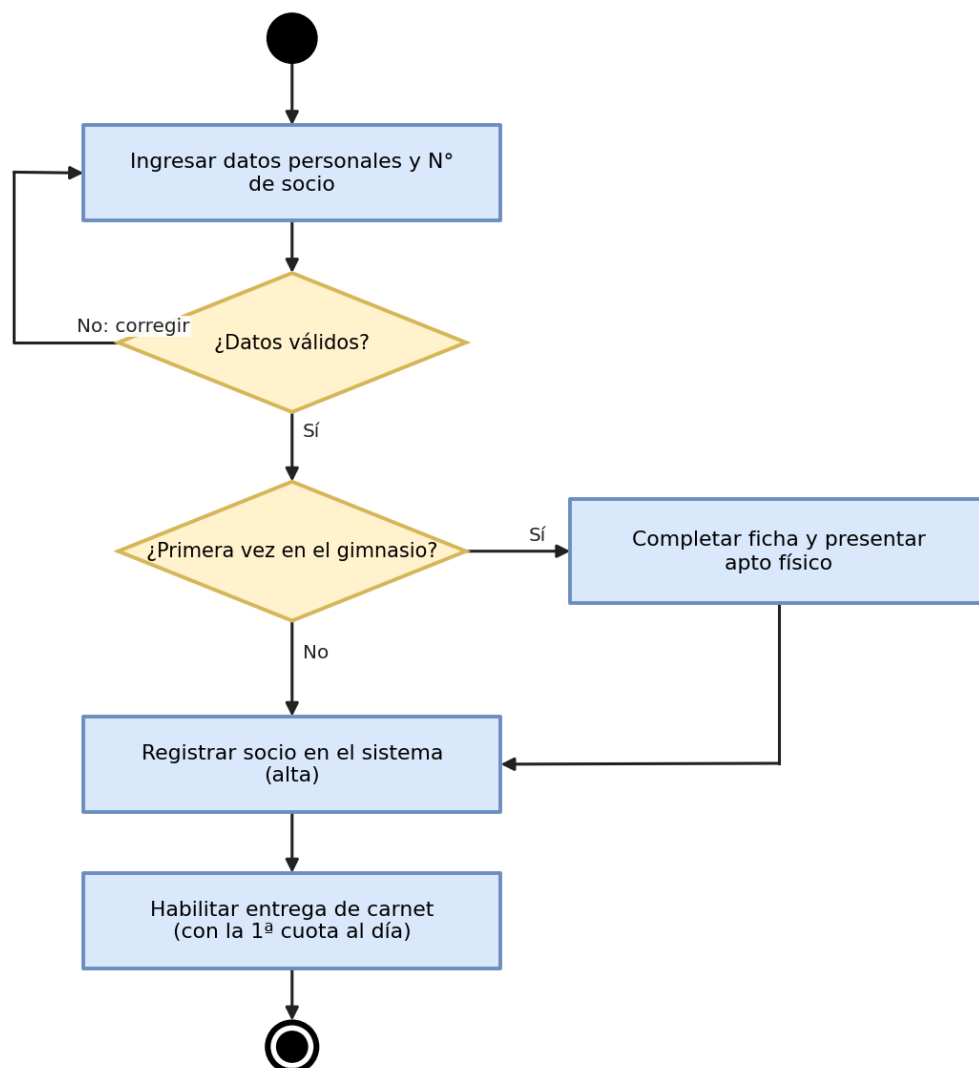


Fig. 7 — Diagrama de Actividades: Alta de Socio

6. Juego de prueba

Casos de prueba funcionales, con datos de entrada y el resultado esperado según las reglas de negocio implementadas. Ejecutados sobre la base Proyecto con los datos de ejemplo del script SQL (fecha de referencia: 28/09/2026).

N°	Descripción	Datos de entrada	Resultado esperado	Resultado obtenido
CP01	Iniciar sesión con credenciales válidas	usuario=admin, contraseña=admin123	Ingresa a frmMenuPrincipal con el menú completo de Administrador	Igual al esperado ✓
CP02	Iniciar sesión con contraseña incorrecta	usuario=admin, contraseña=incorrecta	Mensaje 'Usuario o contraseña incorrectos', permanece en frmLogin	Igual al esperado ✓
CP03	Alta de socio con datos válidos	Nombre=Pedro, Apellido=Ramírez, DNI=30999888, N°Socio=S-0005, aptoFísico=Si	Socio creado; aparece en la grilla con estado 'Activo'	Igual al esperado ✓
CP04	Alta de socio con DNI duplicado	DNI=30111222 (ya existe)	Excepción de negocio: 'Ya existe una persona registrada con ese DNI'	Igual al esperado ✓
CP05	Cobro de cuota en efectivo a socio con cuota al día	Socio S-0001 (vence 15/10/2026), monto=15000, forma=Efectivo	Nuevo vencimiento = 15/11/2026 (día siguiente al vencimiento anterior + 1 mes)	Igual al esperado ✓
CP06	Cobro de cuota con tarjeta en 6 cuotas	Socio S-0004, monto=15000, forma=Tarjeta, cuotas=6	Cuota registrada con formaPago='Tarjeta' y cuotasTarjeta=6	Igual al esperado ✓
CP07	Listado diario de vencimientos	Fecha=15/10/2026	La grilla muestra al socio S-0001 (Pérez, Juan)	Igual al esperado ✓
CP08	Emitir carnet a socio sin apto físico	Socio S-0004 (aptoFísico=No)	Botón 'Emitir carnet' deshabilitado / excepción 'debe presentar el apto físico'	Igual al esperado ✓
CP09	Emitir carnet a socio con apto físico y cuota al día	Socio S-0001	Carnet emitido; socio.carnetEntregado=true	Igual al esperado ✓
CP10	Alta de profesor suplente	Nombre=Ana, Apellido=Ríos, Legajo=P-102, Tipo=Suplente	Profesor creado; aparece al filtrar 'Solo suplentes'	Igual al esperado ✓
CP11	Firmar asistencia dos veces el mismo día	Profesor P-100, firma repetida	Excepción: 'El profesor ya registró su asistencia hoy'	Igual al esperado ✓
CP12	Liquidar sueldo de un profesor	Profesor P-100, mes=9, año=2026, monto=450000	Sueldo insertado/actualizado con fecha_pago = hoy	Igual al esperado ✓
CP13	Asignar turno de nutrición en día distinto a miércoles	Fecha = jueves	Excepción: 'El servicio de nutrición solo atiende los días miércoles'	Igual al esperado ✓
CP14	Asignar turno de nutrición un miércoles válido	Socio S-0001, fecha=miércoles, turno=3	Turno registrado y visible en el historial del socio	Igual al esperado ✓

Resultado obtenido: en las 14 pruebas ejecutadas durante el desarrollo, el comportamiento coincidió con el esperado (columna anterior). Se recomienda volver a correr este juego de pruebas tras cada cambio relevante en la capa de Negocio o en los stored procedures.

7. Conclusiones del desarrollo

El desarrollo del sistema permitió recorrer el ciclo de vida completo de un proyecto de software — análisis, diseño, programación, prueba y puesta a punto — aplicado a un caso realista con reglas de negocio no triviales (vencimiento de cuotas con reglas de fecha, promociones de pago, control de acceso por rol, turnos semanales).

La **arquitectura en capas** facilitó aislar los cambios: por ejemplo, la regla de cálculo del nuevo vencimiento vive únicamente en `sp_CobrarCuota` (base de datos) y en `CuotaNegocio` (validaciones), sin tocar ningún formulario. Concentrar todo el acceso a datos en la carpeta `Datos/`, con una única clase `Conexion` y un DAO por entidad, simplificó además el control de errores (un solo lugar donde se abre y cierra la conexión) y la prevención de inyección SQL (consultas siempre parametrizadas).

Como mejoras posibles a futuro se identifican: registrar un **log de auditoría** de altas/bajas/modificaciones; enviar **recordatorios automáticos** (email/SMS) antes del vencimiento de la cuota; permitir **recuperación de contraseña**; e integrar una pasarela de pago real para el cobro con tarjeta.

8. Instrucciones paso a paso

8.1 Crear el proyecto en Visual Studio

Este entregable ya incluye el proyecto armado: basta con abrir `ClubDeportivo.sln` con Visual Studio 2019 o superior y compilar. El `.csproj` usa el formato **clásico**, que es el que mejor soporta Visual Studio 2019 y permite abrir cada formulario en la **vista de Diseñador**.

1. Abrir `ClubDeportivo.sln` con Visual Studio → se restauran solos los paquetes NuGet (`MySQL.Data`) al compilar.
2. Si se prefiere crear el proyecto desde cero: **Crear un nuevo proyecto** → plantilla **Aplicación de Windows Forms (.NET Framework)** → `.NET Framework 4.8`, y luego copiar dentro las carpetas `Entidades/Datos/Negocio/Formularios` de este entregable.
3. Verificar en Propiedades del proyecto que la plataforma de destino sea Any CPU y el tipo de salida Aplicación de Windows.

8.2 Crear la carpeta "Datos" dentro del proyecto

1. En el **Explorador de soluciones**, clic derecho sobre el proyecto **ClubDeportivo**.
2. Seleccionar **Agregar** → **Nueva carpeta**.
3. Nombrarla **Datos**.
4. Repetir para **Entidades**, **Negocio** y **Formularios**.

8.3 Instalar el conector MySQL (NuGet MySql.Data)

1. Clic derecho sobre el proyecto → **Administrar paquetes NuGet...**
2. Pestaña **Examinar**, buscar **MySql.Data** (autor Oracle/MySQL).
3. Instalar la versión estable más reciente compatible con `.NET Framework 4.8`.

8.4 Configurar la base de datos

1. Instalar **XAMPP** o **MySQL Server + Workbench** y verificar que el servicio MySQL esté iniciado (puerto 3306 por defecto).
2. Abrir `Database/proyecto_club.sql` con Workbench, HeidiSQL o `mysql -u root -p < proyecto_club.sql`, y ejecutarlo completo: crea la base **Proyecto**, sus 13 tablas, 4 stored procedures y datos de ejemplo.
3. Si el servidor, usuario o clave de la instalación local difieren, ajustar la cadena de conexión en `ClubDeportivo/App.config` (clave `ClubDeportivoDB`).

8.5 Ejecutar y compilar

1. Presionar **F5** (o `Ctrl+F5`) en Visual Studio para compilar y ejecutar en modo Debug.
2. Iniciar sesión con alguno de los usuarios de prueba: **admin / admin123** (Administrador), **recepccion / recep123** (Recepcionista), **profesor1 / prof123** (Profesor) o **nutricionista1 / nutri123** (Nutricionista).

8.6 Generar el ejecutable (Release) y desplegarlo

1. En la barra de herramientas, cambiar la configuración de **Debug** a **Release**.
2. Menú **Compilar** → **Compilar ClubDeportivo** (o **Publicar** para generar una carpeta lista para copiar).
3. El ejecutable queda en `ClubDeportivo\bin\Release\net48\ClubDeportivo.exe`.
4. Copiar esa carpeta al equipo destino junto con `ClubDeportivo.exe.config`, y asegurarse de que ese equipo tenga acceso a la base MySQL configurada.